

第二章 线性表

目录



2.1 线性表及其逻辑结构

2.2 线性表的顺序存储结构

2.3 线性表的链式存储结构

2.4 线性表的应用

2.5 有序表

线性表的基本概念

2.1.1 线性表的定义

线性表是一个具有相同特性的数据元素的有限序列。



- 一致性：所有元素属于同一数据类型。
- 有穷性：数据元素个数是有限的。
- 序列性：数据元素由逻辑序号唯一确定。一个线性表中可以有相同值的元素。

线性表中所含元素的个数叫做线性表的长度，用 n 表示， $n \geq 0$ 。 $n=0$ 时，表示线性表是一个空表，即表中不包含任何元素。

线性表的基本概念

2.1.1 线性表的定义

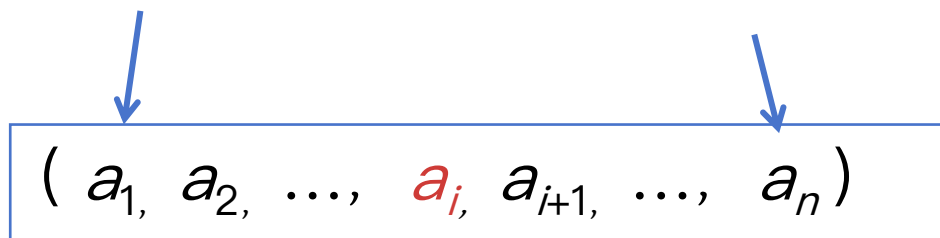
- 线性表的一般表示为:

$$(a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n)$$

a_i ($1 \leq i \leq n$) 表示第 i (i 表示逻辑位序) 个元素。

表头元素

表尾元素



线性表的基本概念

2.1.1 线性表的定义

- 线性表的图形表示为:



- 线性表的二元组表示为: $L=(D,R)$

$$D=\{a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_n\}$$

$$R=\{<a_1, a_2>, <a_2, a_3>, \dots, <a_{n-1}, a_n>\}$$

线性表的基本概念

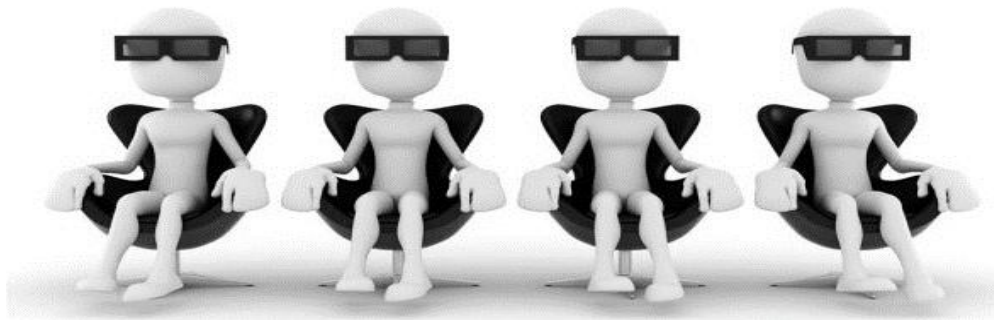
2.1.1 线性表的定义

线性表是客观事物的抽象

① 一个汽车线性表



② 一个小人线性表



.....不胜枚举



线性表的基本概念



2.1.2 线性表的运算

线性表的抽象数据类型描述:

ADT List

{

数据对象:

$D = \{ a_i \mid 1 \leq i \leq n, n \geq 0, a_i \text{ 为 ElemType 类型} \}$

数据关系:

$R = \{ \langle a_i, a_{i+1} \rangle \mid a_i, a_{i+1} \in D, i = 1, \dots, n-1 \}$

基本运算:

9个基本运算

}



线性表的基本概念



2.1.2 线性表的运算

线性表的基本运算描述:

- ① 初始化线性表 `InitList(&L)`: 构造一个空的线性表 `L`。
- ② 销毁线性表 `DestroyList(&L)`: 释放线性表 `L` 占用的内存空间。
- ③ 判线性表是否为空表 `ListEmpty(L)`: 若 `L` 为空表, 则返回真, 否则返回假。
- ④ 求线性表的长度 `ListLength(L)`: 返回 `L` 中元素个数 n 。



线性表的基本概念



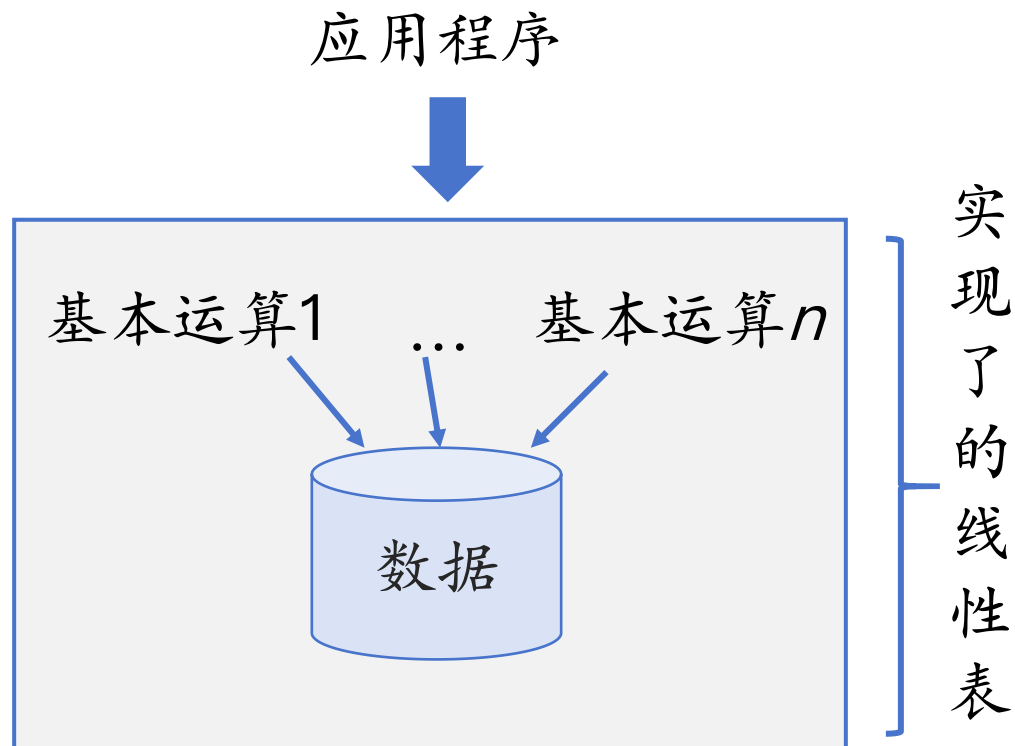
2.1.2 线性表的运算

- ⑤ 输出线性表DispList(L): 线性表L不为空时, 顺序显示L中各结点的值域。
- ⑥ 求线性表L中指定位置的某个数据元素GetElem(L, i, &e): 用e返回L中第 i ($1 \leq i \leq n$) 个元素的值。
- ⑦ 定位查找LocateElem(L, e): 返回L中第一个值域与e相等的逻辑位序。若这样的元素不存在, 则返回值为0。
- ⑧ 插入一个数据元素ListInsert(&L, i, e): 在L的第i ($1 \leq i \leq n$) 个元素之前插入新的元素e, L的长度增1。
- ⑨ 删除数据元素ListDelete(&L, i, &e): 删除L的第i ($1 \leq i \leq n$) 个元素, 并用e返回其值, L的长度减1。

线性表的基本概念

2.1.2 线性表的运算

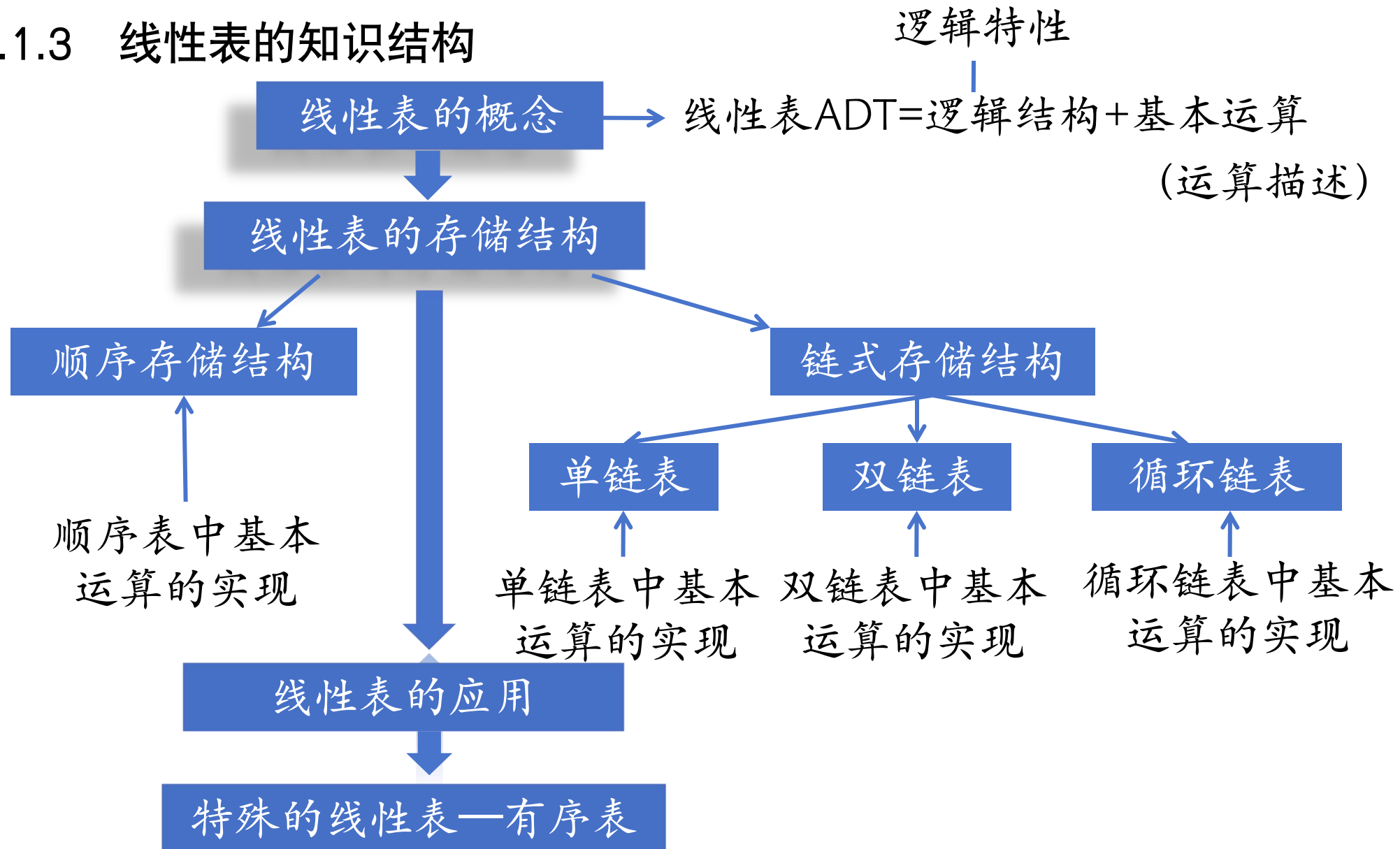
线性表的作用



- 程序员可以直接使用它来存放数据——作为存放数据的容器。
- 程序员可以直接使用它的基本运算——完成更复杂的功能。

线性表的基本概念

2.1.3 线性表的知识结构



线性表的基本概念

2.1.3 线性表的知识结构

线性表重要的知识点:

- 线性表两类存储结构的差异。
- 每种存储结构中基本运算的实现算法。
- 利用线性表求解实际问题。
- 利用有序表特性设计高效算法。

目录

2.1 线性表及其逻辑结构



2.2 线性表的顺序存储结构

2.3 线性表的链式存储结构

2.4 线性表的应用

2.5 有序表

线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

线性表的顺序存储结构:

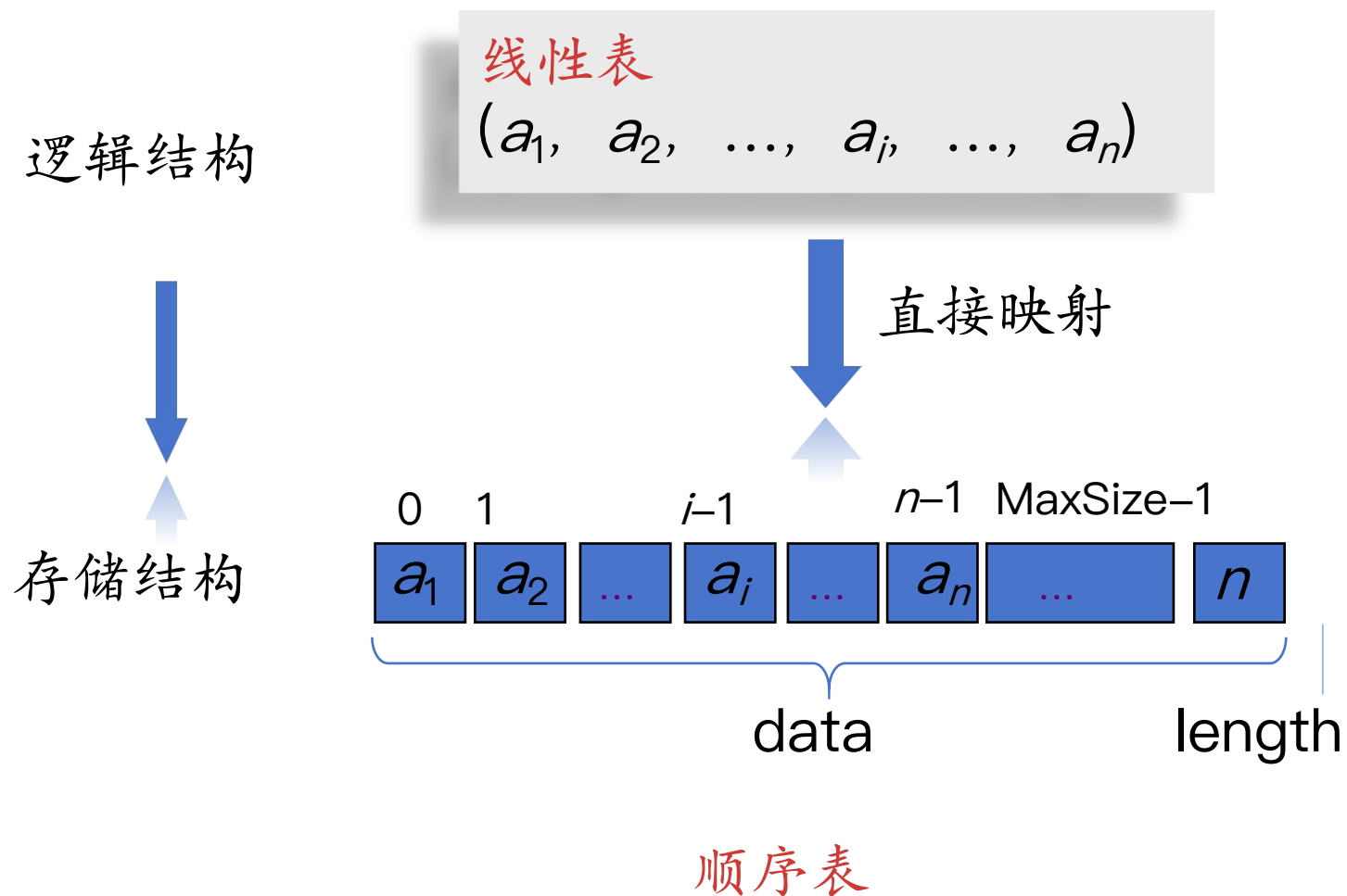
把线性表中的所有元素按照顺序存储方法进行存储。



按逻辑顺序依次存储到存储器中一片连续的存储空间中。

线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表



线性表的顺序存储结构

2.2.1 线性表的顺序存储—顺序表

➤ 顺序表类型定义

这里，假设ElemType为char类型

```
typedef struct
{ ElemType data[MaxSize];
  int length;
} SqList;    //顺序表类型
```

其中data成员存放元素，length成员存放线性表的实际长度。

说明： 注意逻辑位序和物理位序相差1。



线性表的顺序存储结构



2.2.2 顺序表运算的实现

1、建立顺序表

$a[0..n-1] \Rightarrow$ 顺序表L — 整体创建顺序表。

```
void CreateList(SqList * &L, ElemType a[], int n)
{ int i=0,k=0;
  L=(SqList *)malloc(sizeof(SqList));
  while (i<n) //i扫描a中元素
  { L->data[k]=a[i];
    k++; i++; //k记录插入到L中的元素个数
  }
  L->length=k;
}
```

传递顺序表
指针

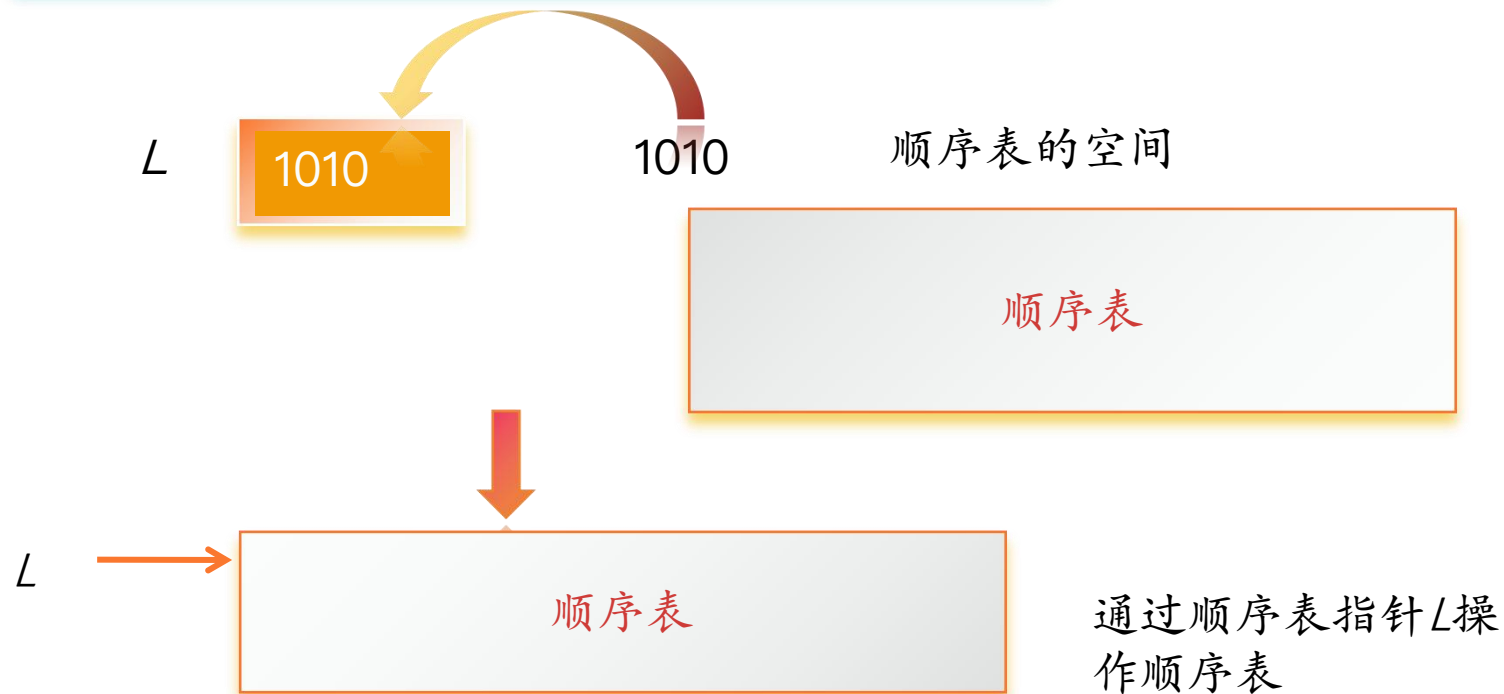
线性表的顺序存储结构

2.2.2 顺序表运算的实现

算法参数说明

① 顺序表指针的含义

```
SqList *L;  
L=(SqList *)malloc(sizeof(SqList));
```



线性表的顺序存储结构

2.2.2 顺序表运算的实现

② 顺序表指针引用

```
void CreateList(SqList *&L, ElemType a[], int n)
```



引用参数：将执行结果回传给实参

- 引用符号“&”放在形参L的前面。
- 输出型参数均为使用“&”，不论参数值是否改变。



线性表的顺序存储结构



2.2.2 顺序表运算的实现

2、顺序表基本运算算法

(1) 初始化线性表InitList(L)

构造一个空的线性表L。实际上只需将length成员设置为0即可。

```
void InitList(SqList *&L)
{ L=(SqList *)malloc(sizeof(SqList));
  //分配存放线性表的顺序表空间
  L->length=0;
}
```



线性表的顺序存储结构



2.2.2 顺序表运算的实现

(2) 销毁线性表DestroyList(L)

释放线性表L占用的内存空间。



free(L)释放L所指向的空间

顺序表采用指针传递，有两个优点：

- 更清楚看到顺序表创建和销毁过程 (malloc/free) 。
- 在算法的函数之间传递更加节省空间 (在函数体内不必创建值形参即整个顺序表的副本) 。



线性表的顺序存储结构



2.2.2 顺序表运算的实现

(3) 判定是否为空表ListEmpty(L)

回一个值表示L是否为空表。若L为空表，则返回true，否则返回false。

```
bool ListEmpty(SqList *L)
{
    return(L->length==0);
}
```



线性表的顺序存储结构



2.2.2 顺序表运算的实现

(4) 求线性表的长度ListLength(L)

返回顺序表L的长度。实际上只需返回length成员的值即可。

```
int ListLength(SqList *L)
{
    return(L->length);
}
```



线性表的顺序存储结构



2.2.2 顺序表运算的实现

(5) 输出线性表DispList(L)

当线性表L不为空时，顺序显示L中各元素的值。

```
void DispList(SqList *L)
{ int i;
  if (ListEmpty(L)) return;
  for (i=0;i<L->length;i++)
    printf("%c", L->data[i]);
  printf("\n");
}
```




线性表的顺序存储结构



2.2.2 顺序表运算的实现

(6) 求某个数据元素值 `GetElem(L, i, e)`

返回L中第 i ($1 \leq i \leq \text{ListLength}(L)$) 个元素的值, 存放在 e 中。

```
bool GetElem(SqList *L, int i, ElemType &e)
{
    if (i < 1 || i > L->length) return false;
    e = L->data[i-1];
    return true;
}
```

本算法的时间复杂度为 $O(1)$ 。



体现顺序表的**随机存取特性**



线性表的顺序存储结构



2.2.2 顺序表运算的实现

(7) 按元素值查找LocateElem(L, e)

顺序查找第一个值域与 e 相等的元素的逻辑位序。若这样的元素不存在，则返回值为0。

```
int LocateElem(SqList *L, ElemType e)
{ int i=0;
  while (i<L->length && L->data[i]!=e)
    i++;
  if (i>=L->length) return 0;
  else return i+1;
}
```



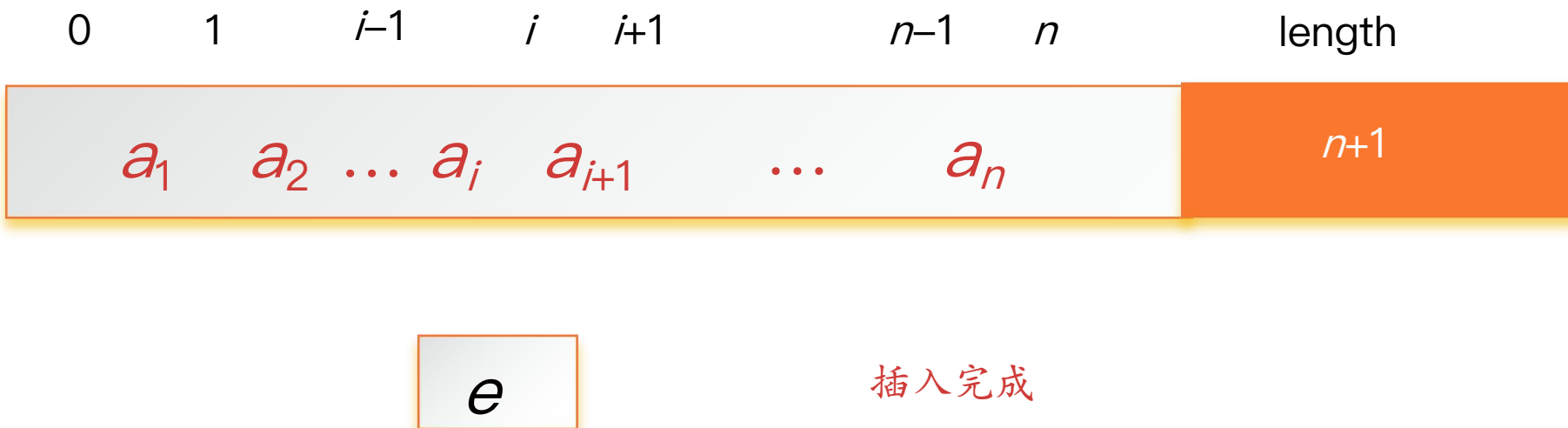
线性表的顺序存储结构



2.2.2 顺序表运算的实现

(8) 插入数据元素 $\text{ListInsert}(L, i, e)$

在顺序表 L 的第 i ($1 \leq i \leq \text{ListLength}(L)+1$) 个位置上插入新的元素 e 。





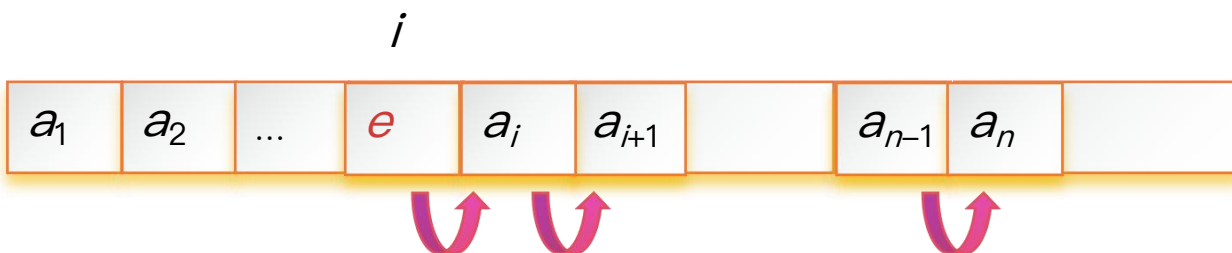
线性表的顺序存储结构



2.2.2 顺序表运算的实现

插入算法如下:

```
bool ListInsert(SqList *&L, int i, ElemType e)
{
    int j;
    if (i < 1 || i > L->length + 1)
        return false;           // 参数错误时返回false
    i--;                          // 将顺序表逻辑序号转化为物理序号
    for (j = L->length; j > i; j--) // 将data[i..n]元素后移一个位置
        L->data[j] = L->data[j-1];
    L->data[i] = e;               // 插入元素e
    L->length++;                  // 顺序表长度增1
    return true;                 // 成功插入返回true
}
```





线性表的顺序存储结构



2.2.2 顺序表运算的实现

对于本算法来说，元素移动的次数不仅与表长 $L \rightarrow \text{length} = n$ 有关，而且与插入位置 i 有关：

- 当 $i = n + 1$ 时，移动次数为 0；
- 当 $i = 1$ 时，移动次数为 n ，达到最大值。

算法最好时间复杂度为 $O(1)$

算法最坏时间复杂度为 $O(n)$

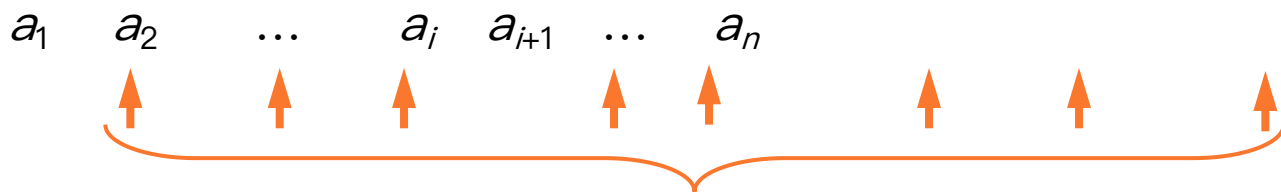


线性表的顺序存储结构



2.2.2 顺序表运算的实现

平均情况分析:



在线性表L中共有 $n+1$ 个可以插入元素的地方

在插入元素 a_i 时, 若为等概率情况, 则 $p_i = \frac{1}{n+1}$

此时需要将 $a_i \sim a_n$ 的元素均后移一个位置, 共移动 $n-i+1$ 个元素。

在长度为 n 的线性表中插入一个元素时所需移动元素的平均次数为:

$$\sum_{i=1}^{n+1} p_i (n - i + 1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n - i + 1) = \frac{n}{2}$$

因此插入算法的平均时间复杂度为 $O(n)$ 。



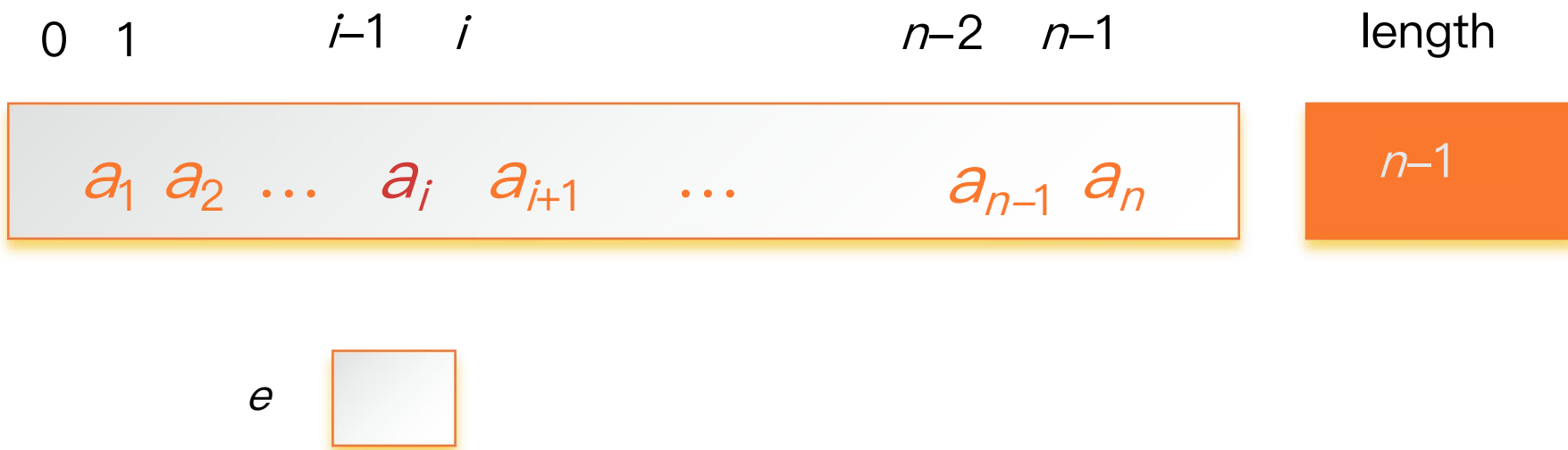
线性表的顺序存储结构



2.2.2 顺序表运算的实现

(9) 删除数据元素 $ListDelete(L, i, e)$

删除顺序表 L 的第 i ($1 \leq i \leq ListLength(L)$) 个元素。



删除完成



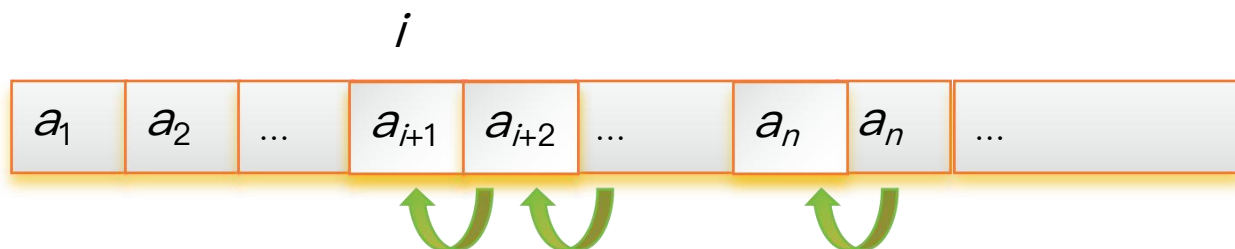
线性表的顺序存储结构



2.2.2 顺序表运算的实现

删除算法如下:

```
bool ListDelete(SqList *&L, int i, ElemType &e)
{ int j;
  if (i<1 || i>L->length)           //参数错误时返回false
    return false;
  i--;                               //将顺序表逻辑序号转化为物理序号
  e=L->data[i];
  for (j=i;j<L->length-1;j++)        //将data[i..n-1]元素前移
    L->data[j]=L->data[j+1];
  L->length--;                        //顺序表长度减1
  return true;                       //成功删除返回true
}
```





线性表的顺序存储结构



2.2.2 顺序表运算的实现

对于本算法来说，元素移动的次数也与表长 n 和删除元素的位置 i 有关：

- 当 $i=n$ 时，移动次数为0；
- 当 $i=1$ 时，移动次数为 $n-1$ 。

删除算法最好时间复杂度为 $O(1)$

删除算法最坏时间复杂度为 $O(n)$

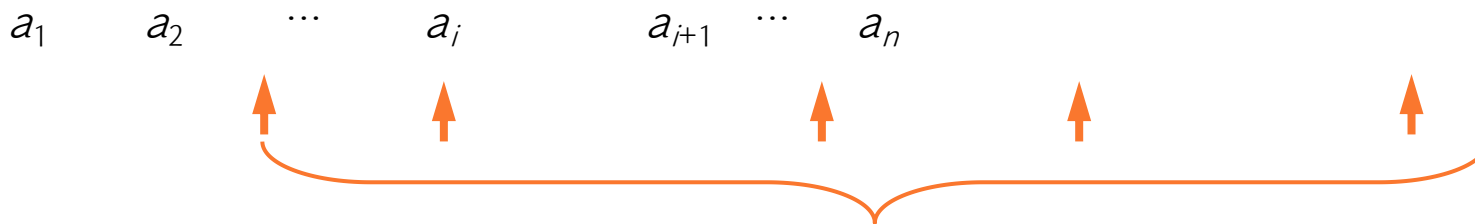


线性表的顺序存储结构



2.2.2 顺序表运算的实现

平均情况分析:



在线性表L中共有 n 个可以删除元素的地方

在删除元素 a_i 时, 若为等概率情况, 则 $p_i = \frac{1}{n}$

此时需要将 $a_{i+1} \sim a_n$ 的元素均前移一个位置, 共移动 $n-(i+1)+1=n-i$ 个元素。

所以在长度为 n 的线性表中删除一个元素时所需移动元素的平均次数为:

$$\sum_{i=1}^n p_i (n-i) = \sum_{i=1}^n \frac{1}{n} (n-i) = \frac{n-1}{2}$$

因此删除算法的平均时间复杂度为 $O(n)$ 。